

Phân đoạn tiền cảnh thời gian thực và làm mờ có chọn lọc trên GPU

Huỳnh Lê Hải Dương (22127081) Nguyễn Đức Tín (22127415)

CSC14116 - Applied Parallel Programming, HCMUS

Giới thiệu

Phương pháp

Đánh giá

Thảo luận

Kết luận

- ▶ **Input:** video giao thông camera cố định (BGR, 320×240 tới 1920×1080)
- ▶ **Output mỗi frame:** mask nhị phân (255 = xe đang chạy) và frame composite - xe nét, nền mờ
- ▶ **Goal:** thời gian thực ≥ 30 FPS ở 1080p; chất lượng chấm trên nhãn tay CDnet highway (frame 470–1700)



ground truth (0 background - 50 bóng - 170 chưa biết - 255 foreground)



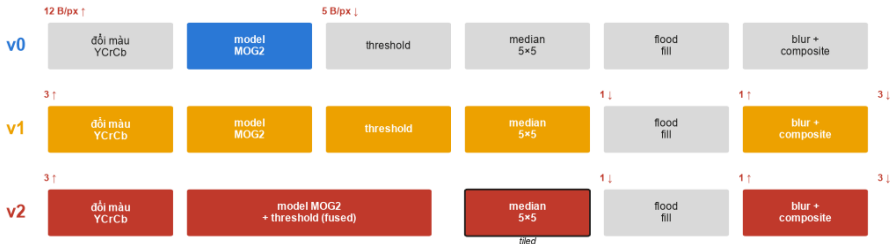
- ▶ 1080p có **2 073 600 pixel**, mỗi pixel cập nhật mô hình *độc lập* $\Rightarrow$ một frame ánh xạ thẳng thành 2 triệu thread
- ▶ Sequential Python: **3 035 ms/frame** ở 480p - còn mục tiêu là 30 FPS
- ▶ Nhưng không đưa tất cả lên GPU: một giai đoạn được *giữ lại trên CPU* có chủ đích (phần Phương pháp)

- ▶ Mỗi pixel giữ **5 Gaussian** - 5 “màu quen thuộc” của nền, kèm phương sai và trọng số
- ▶ $\text{dist}^2 < T_b \cdot \text{var}$ ($T_b=16$): fit lỏng → cộng bằng chứng background
- ▶ $\text{dist}^2 < T_g \cdot \text{var}$ ($T_g=9$): fit chặt → cập nhật mode, tốc độ học $\alpha = 1/250$; không mode nào fit → thay mode yếu nhất
- ▶ Tổng trọng số tới $TB=0.9$ tạo thành mô hình nền; foreground khi $\text{bg_prob} < 0.5$
- ▶ Cùng thuật toán, cùng tham số cho **cả 5 bản cài đặt** ⇒ so sánh tốc độ là công bằng

- ▶ **Sequential Python** - bản đặc tả dễ đọc · **Numba CPU** - baseline đã biên dịch
- ▶ **CUDA v0 (L1)**: model kernel 1 thread/pixel, block 32×8 khớp warp, state planar coalesced. Mọi thứ khác còn ở host, upload 12 B/px
- ▶ **CUDA v1 (L2 - bộ nhớ)**: upload 3 B/px; YCrCb, threshold, median, blur, composite thành kernel; mask ở lại device tới bước fill
- ▶ **CUDA v2 (L3 - compute)**: fuse threshold vào model kernel (3 kernel còn 2, trung gian 1 B/px); median + blur trên shared-memory tile

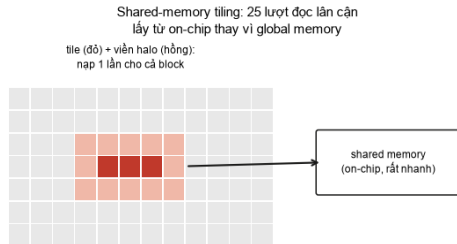
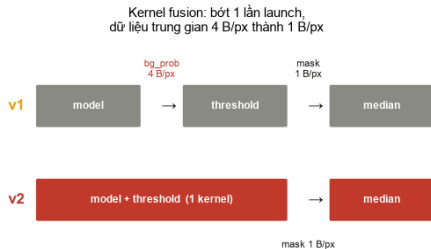
Giai đoạn nào chạy ở đâu

Xám = chạy trên CPU (host) · Màu = chạy trên GPU (device) · số đỏ = byte/pixel qua bus PCIe

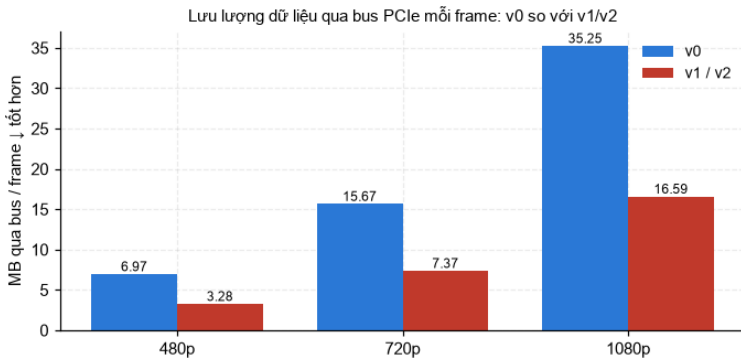


- v1 chuyển gần hết pipeline sang GPU; v2 gộp kernel và tile hóa; flood fill ở lại CPU trong mọi bản

Hai kỹ thuật đặc trưng của v2



v1 cắt lưu lượng bus còn một nửa



- ▶ 1080p: **35.25 → 16.59 MB/frame** - lợi ích lớn nhất của v1 là *di chuyển ít dữ liệu hơn*

Phát hiện bất ngờ: blur của OpenCV là toán số nguyên

Cả hai kernel gồm 15 số nguyên có tổng 256 - lệch ± 1 tại 4 tap, và chỉ một kernel khớp cv2 (đỏ = pixel khác, đã phóng lớn)

làm tròn từng tap - phỏng đoán tự nhiên
15,617 px khác cv2

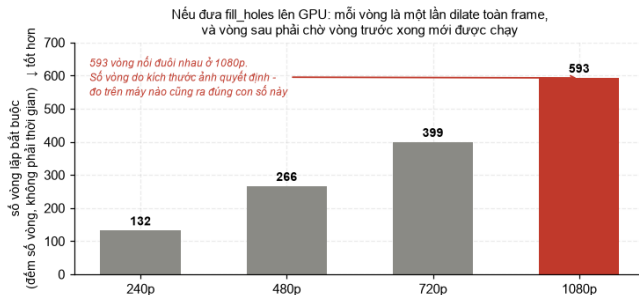


tích lũy - cách của nhóm, theo OpenCV
0 px khác - bit-exact



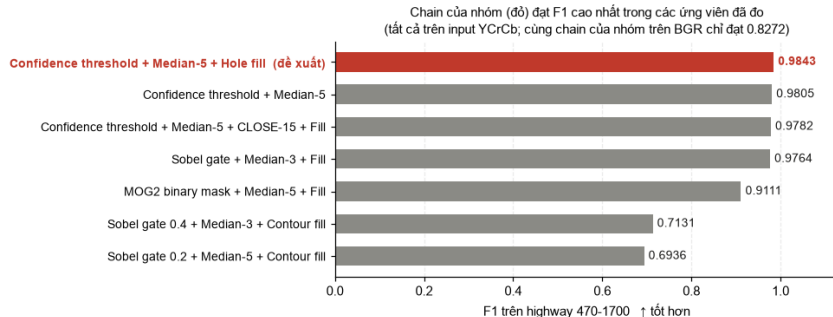
- ▶ Làm tròn từng hệ số: **15,617 px lệch** cv2 · lượng tử hóa tích lũy Q8 (nhóm tái tạo): **0 px**
- ▶ Số nguyên $\Rightarrow$ đúng *tuyệt đối* trên mọi compiler - float32 không hứa được điều đó

Giai đoạn giữ trên CPU: flood fill



- ▶ Bản GPU cần **593 vòng dilate toàn frame nối đuôi nhau** ở 1080p - đếm số vòng, không phải thời gian
- ▶ Biết giai đoạn nào **không** nên chuyển lên GPU cũng là một kết quả (Partial GPU Principle)

Chọn chuỗi hậu xử lý bằng phép đo

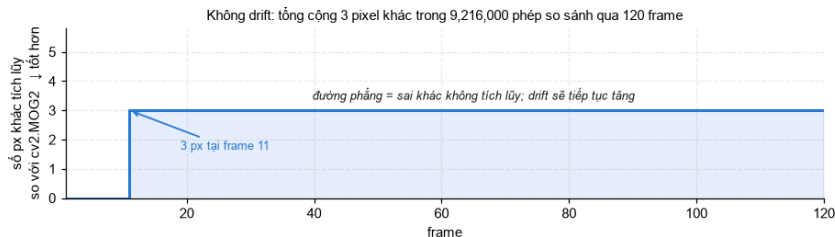


- Chain của nhóm dẫn đầu: **F1 0.9843**; cùng chain trên BGR chỉ 0.8272 - không gian màu quan trọng nhất

Parity: 5 bản cài đặt, một kết quả

- ▶ Frame tổng hợp: mask khớp **từng bit** giữa cả 5 bản · **75 test** xanh trên T4
- ▶ 1231 frame thật: **2 pixel lệch trên 94.5 triệu** - và gate chứng minh *từng pixel*:
 - ▶ một pixel nằm ngay ngưỡng 0.5 (0.4999971 so với 0.5000015)
 - ▶ một pixel truy về đúng một phép so sánh bị hai compiler làm tròn lệch *1 ulp*
- ▶ Kết luận: ranh giới số thực float32, **không phải bug**; mọi giai đoạn số nguyên khớp tuyệt đối

Không trôi khỏi OpenCV theo thời gian



- ▶ Nhảy **3 px đúng một lần** ở frame 11 rồi phẳng suốt 110 frame - lệch không tích lũy

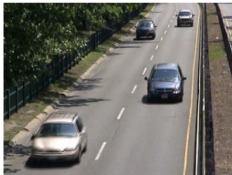
Chất lượng mask trực quan



- ▶ Trắng = khớp nhãn · cam = báo thừa · đỏ = bỏ sót - lỗi chỉ nằm ở *viền xe và bóng*

So trực tiếp với thư viện và nhãn

đầu vào



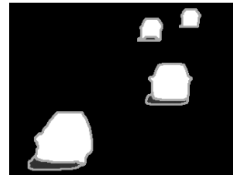
chain của nhóm (Numba, đại diện)



cv2 MOG2, raw (không post chain)

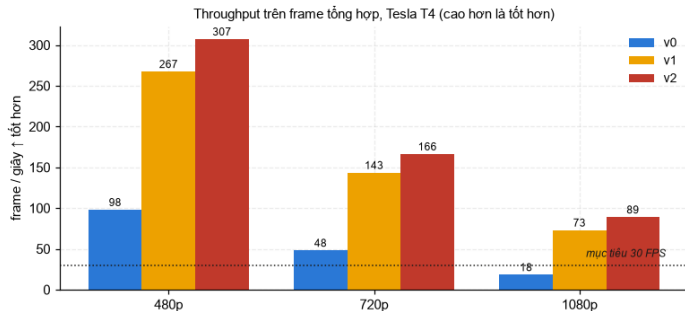


ground truth



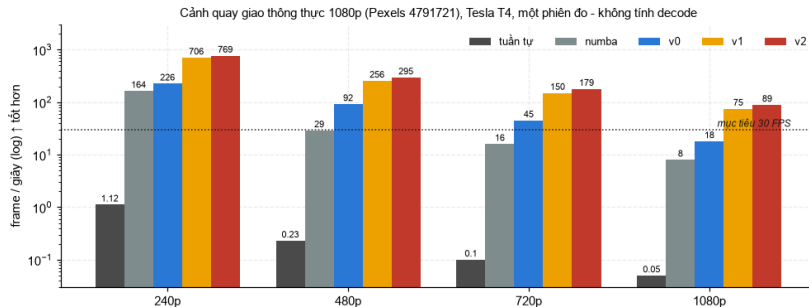
frame 1100 - chỉ kết quả nhóm được áp post chain; cv2 hiển thị raw

Thông lượng trên frame tổng hợp



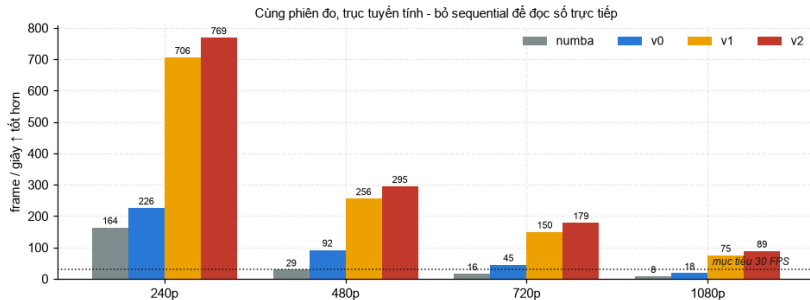
- v2: **88.8 FPS** ở 1080p = $4.89 \times$ v0, gấp 2.9 lần mục tiêu · target $>20 \times$ vs sequential chốt ở **887x**

Cảnh quay thực: đủ 5 bản, một phiên đo

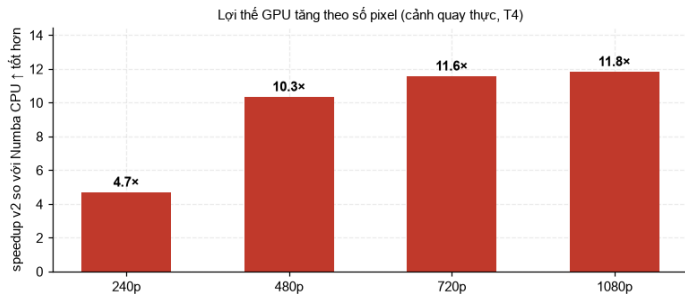


- ▶ v2 giữ **89 FPS** ở 1080p thật; sequential 0.05 FPS - nhanh hơn khoảng **1933x** trên cùng clip

Cùng dữ liệu, trực tuyến tính

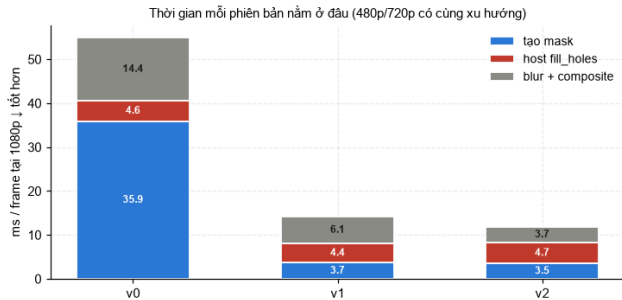


Lợi thế GPU tăng theo số pixel



- ▶ v2 so với Numba CPU: **4.7x ở 240p → 11.8x ở 1080p** - lớn nhất đúng nơi thời gian thực khó nhất

Thời gian mỗi frame nằm ở đâu



- Model và blur giảm mạnh; host flood fill đứng yên → **39.7%** frame của v2 - bước tiếp theo là CUDA streams

Kết quả đầu cuối



nguồn: CDnet highway

0.9843

F1 trên CDnet highway

1231 frame - record đã commit - ↑ tốt hơn

88.8

FPS tại 1080p trên T4

benchmark tổng hợp - mục tiêu 30 - 4.89× v0 - ↑

2

px khác trên 94.5 M

trên T4 - float-boundary đã chứng minh - ↓ tốt hơn

4

bản build OpenCV

blur bit-exact trên cả bốn bản

Sáu dự đoán ghi trước khi đo: 5 sai, 1 đúng

Dự đoán	Ghi trước	Đo được	
Host blur + composite @1080p	4–8 ms	13.7 ms	Sai
GPU blur kernels @1080p	0.2–0.4 ms	1.55 ms	Sai
Tiết kiệm ingest + blur @1080p	4–8 ms	~34 ms	Sai
Giảm lưu lượng bus	–38.5%	–52.9%	Sai
Tiling so với naive	“xấp xỉ naive”	nh nhanh hơn 2.36x	Sai
Bottleneck sau thay đổi	host fill	đúng - 39.7%	Đúng

- ▶ Giữ nguyên dự đoán sai trong báo cáo: dự đoán ghi *trước*, số đo đến *sau* - phần giá trị là giải thích vì sao lệch

- ▶ Bottleneck ban đầu ở **host và di chuyển dữ liệu**, không phải phép tính
- ▶ Chọn được **số nguyên** ở đâu là mua được sự đúng tuyệt đối ở đó (blur Q8)
- ▶ Có giai đoạn **thuộc về CPU** - 593 vòng phụ thuộc không biến mất dù kernel nhanh gấp trăm lần

- ▶ Một thuật toán, năm bản cài đặt, mọi khẳng định đều có test
- ▶ Chất lượng **F1 0.9843** · tốc độ **88.8 / 89 FPS** 1080p (tổng hợp / thực) - cả hai mục tiêu đề xuất đã chốt
- ▶ Sai khác GPU–CPU bị chặn ở **2 pixel đã chứng minh** trên 94.5 triệu
- ▶ **Hạn chế:** mirror dataset, float khác compiler (đã chặn), biến động Colab, không có nhãn 1080p
- ▶ **Hướng tiếp theo:** pinned buffers + CUDA streams (overlap fill với ingest); fused separable blur (trần lợi ích nhỏ, chưa đo nên không claim)

Cảm ơn thầy cô và các bạn!

Nhóm sẵn sàng trả lời câu hỏi.

Repo: `github.com/Sn-cpp/Adaptive-Gaussian-Mixture-Model` - mọi số liệu trace về
record đã commit